Studio Nx

JS 136 — snapOptions()

2026-10-14

JS 136 — `snapOptions()` (스냅 메뉴 목록)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 스물두 번째 함수 (FR-206 그리드 스냅 메뉴)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	스냅 메뉴 선택을 상태에 담는 배선을 진행합니다

1. 하는 일

스냅 메뉴에 넣을 **분할 목록**을 돌려줍니다. `DIVISION_DEN`의 여섯 분할(1/1~1/32)마다 **보통**과 **셋잇단** 두 줄을 만들어, 모두 12줄입니다.

- 셋잇단은 이름 뒤에 `T`를 붙입니다(`'1/16T'`).
- `ticks`는 `snapStepTicks()`로 구합니다(보통 `PPQ×4÷n`, 셋잇단 그 ÷3).
- 순서: 큰 분할부터(1/1), 각 분할은 보통 → 셋잇단.

라벨	division	triplet	ticks
1/1	1/1	false	3840
1/1T	1/1	true	1280
1/2	1/2	false	1920
1/2T	1/2	true	640
…	…	…	…
1/32	1/32	false	120
1/32T	1/32	true	40

2. 코드

```
export function snapOptions() {
  const out = [];
  for (const division of Object.keys(DIVISION_DEN)) {
    out.push({ division, triplet: false, label: division, ticks: snapStepTicks(division, false) });
    out.push({ division, triplet: true, label: division + 'T', ticks: snapStepTicks(division, true) });
  }
  return out;
}
```

최신 문법으로 쓰면 (공부용)

```
export const snapOptions = () =>
  Object.keys(DIVISION_DEN).flatMap((division) => [
    { division, triplet: false, label: division, ticks: snapStepTicks(division, false) },
```

```
    { division, triplet: true, label: `${division}T`, ticks: snapStepTicks(division, true) },
  ]);
```

최신 문법	실제 코드	뜻
`flatMap`	`for` + `push` 두 번	같은 결과입니다. 두 줄을 이어 붙이는 모양이 눈에 보이게 반복으로 씁니다

3. 왜 이렇게

선택	이유
`DIVISION_DEN`을 재사용	분할 이름·분모를 이미 한 곳(116)에서 정의했습니다. 새 목록을 만들면 둘이 어긋날 수 있습니다
보통 + 셋잇단을 한 줄씩	FR-206이 "1/1~1/32, 셋잇단 포함"을 요구합니다. 메뉴에서 바로 고를 수 있게 둘 다 넣습니다
`ticks`까지 함께	메뉴가 tick 값을 바로 알아 끌기 코드에 넘길 수 있습니다. 계산을 메뉴마다 반복하지 않습니다
`label`을 따로	화면에 보일 이름(`'1/16T'`)과 값(`division`·`triplet`)을 나눠, 보이는 글자와 동작을 독립시킵니다
호출마다 새 배열	메뉴를 다시 그릴 때 이전 배열을 건드리지 않습니다

4. 시험 (Node에서 직접 실행, 9개 통과)

확인	결과
줄 수 12 · 순서(보통·셋잇단 교대)	✓
보통 tick [3840,1920,960,480,240,120]	✓
셋잇단 tick [1280,640,320,160,80,40]	✓
`triplet` 표시 · `division` 값	✓
첫 줄·마지막 줄 객체	✓
호출마다 새 배열	✓